

PromptForge

A fully local AI image, video and avatar studio for Windows

Complete documentation, generated from the code on 2026-08-16 (commit 7bfc772).

Regenerate any time: `backend\.venv\Scripts\python.exe docs\generate_documentation.py`

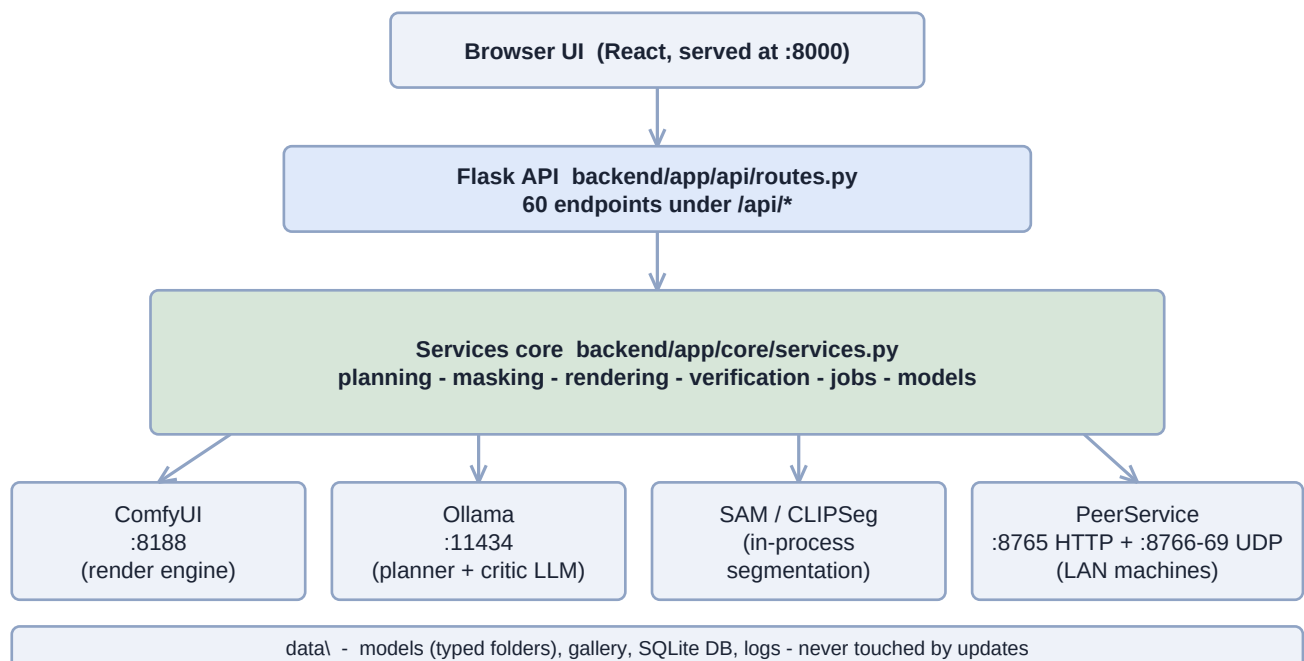
1. What PromptForge is

PromptForge turns plain-language requests — “change the shirt to a red jacket”, “make a video of this photo”, “build a 3D avatar from these pictures” — into real renders on your own hardware. A local LLM plans the work, a segmentation stack finds the exact pixels the words name, ComfyUI executes the render, and vision models judge the result and retry with a better strategy when it falls short. Nothing leaves the machine by default: prompts, photos, models and results all live in the local **data** folder.

Honesty is a design rule, enforced in code: mock output is always labelled, cloud LLM fallback is stamped on every reply that used it, quality scores come from measurements, and every generated image carries a recipe card recording exactly how it was made.

Capabilities (each is a routed pipeline, not a prompt trick): generation (SD1.5 / SDXL / Z-Image / Flux Kontext), region edits with automatic masks, background replacement, relighting, pose change, viewpoint change, style transfer, restoration, upscaling, outpainting, text-to-video and image-to-video (WAN 2.x), motion transfer, multi-image composition, photo-to-3D scenes, and full 3D avatars with photoreal texturing and rigging.

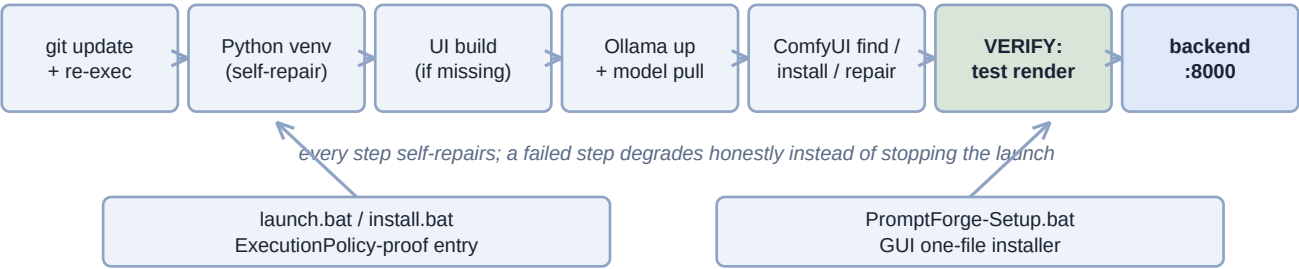
2. System architecture



Sources: `backend/run.py`, `backend/app/api/routes.py`, `backend/app/core/*.py`, `frontend/src/`

One Flask process serves both the built React UI and the JSON API on 127.0.0.1:8000. The Services core owns all intelligence and talks to three engines: ComfyUI (rendering, auto-revived if it dies), Ollama (planning and image critique, auto-revived and auto-pulled), and an in-process segmentation stack (SAM plus a resident CLIPSeg worker running inside ComfyUI's Python). The PeerService is the only network-facing surface and exposes models, renders and logs to other PromptForge machines on the same LAN.

3. Installing and launching



Sources: `launch.ps1`, `launch.bat`, `install.bat`, `installer/*.ps1`

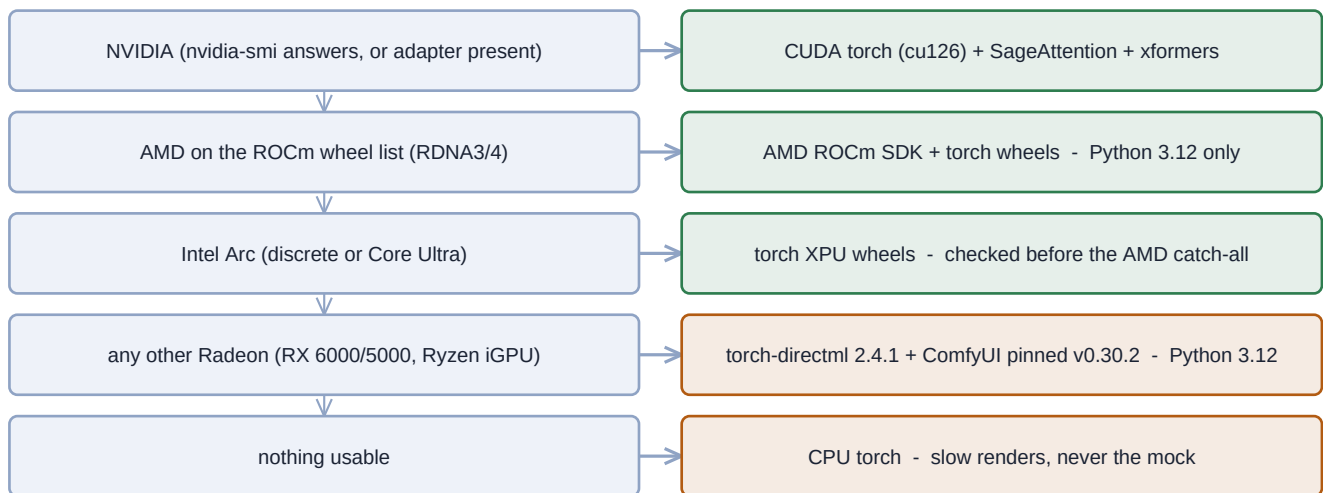
Three equivalent entries, all safe on machines where PowerShell scripts are disabled (each runs with a per-process ExecutionPolicy bypass; nothing changes system settings):

Entry	When to use it
install.bat	After unpacking a GitHub ZIP (connects the folder to the update channel, then launches) — or even as a lone file on an empty PC
launch.bat	Day-to-day start. Runs launch.ps1, which self-repairs everything on every start.
PromptForge-Setup.bat	Single-file GUI installer built by installer\build-installer.ps1 — pick a folder, press Install; no git or Node needed on the target, auto-

launch.ps1 installs anything missing: Python 3.12 (winget, falling back to python.org's own silent installer), Node for the UI build, Ollama (winget or ollama.com), git, ComfyUI itself (git clone first — its schannel TLS survives antivirus certificate interception — then a zip, with retries and full logs), and the right torch for the GPU. A script-level error net turns any remaining failure into one readable message plus the transcript path (`data\logs\launch.log`). The launcher then proves the install: a test graph must actually render before the backend is told ComfyUI is real — otherwise renders fall back to CPU mode, and only when even that fails does the clearly-labelled mock appear.

4. Hardware adaptation

4.1 GPU decision ladder



first match wins - after install every stack is PROBED (torch.cuda / torch.xpu / torch_directml) and reported honestly

Sources: [launch.ps1](#) Get-GpuMode / [Repair-ComfyVenv](#), [installer/installer.ps1](#), [doctor.ps1](#)

4.2 VRAM detection

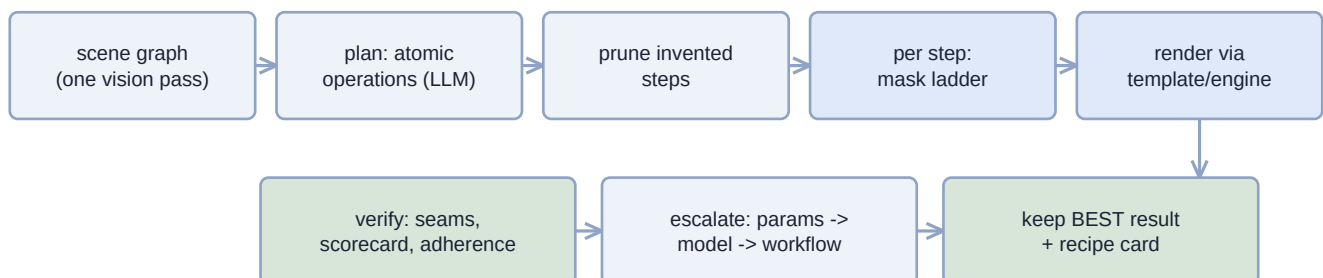
VRAM is read from nvidia-smi when it answers, otherwise from the display-class registry key the GPU driver itself writes (HardwareInformation.qwMemorySize) — so AMD and Intel cards report their true memory instead of 0 GB. The same probe runs in the launcher, the doctor and the backend (hardware.py), and feeds the planner-model choice, render budgets and the peer dashboard.

4.3 What the numbers decide

Tier	VRAM	Max render	Max steps	Video frames	Planner LLM
high	>= 16 GB	1536 px	60	81 @ 768 px	qwen2.5:14b (GPU or >= 32 GB RAM)
mid	>= 6 GB	1280 px	50	81 @ 768 px	qwen2.5:7b (with >= 12 GB RAM)
low	< 6 GB	768 px	40	33 @ 512 px	qwen2.5:3b (7b when RAM >= 32 GB)

Source: [backend/app/core/hardware.py](#) (tier, render_budget, llm_model_for) and the launcher's planner picker. Budgets both inform the planner and CLAMP generated graphs, so an over-ambitious workflow cannot hard-crash the GPU. Machines with 20 GB RAM or less also drop ComfyUI's checkpoint cache between renders — measured as the difference between working video renders and OOM crashes on a 16 GB machine.

5. The edit pipeline



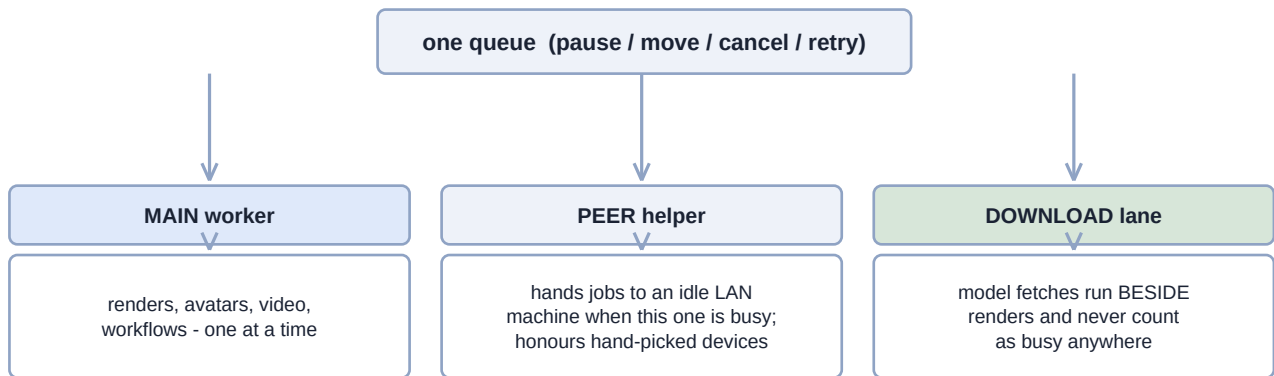
mask ladder: named body/clothing parts -> text-grounded (CLIPSeg worker) -> geometric SAM, each verified deterministically
a mask that finds nothing is reported as 'not found here' - the app never silently edits the wrong pixels
retries always re-run from the last step's input and can never undo the requested edit

Sources: *backend/app/core/services.py* (*_handle_image_edit*), *quality.py* (*plan_edit*, *mask_verdict*, *scorecard*), *scene_graph.py*

Every request is compiled to atomic operations (ADD_OBJECT, REMOVE_OBJECT, REPLACE_OBJECT, CHANGE_ATTRIBUTE, CHANGE_STYLE, CHANGE_LIGHTING, CHANGE_POSE, CHANGE_CAMERA, CHANGE_TEXT, COMPOSE, OUTPAINT, UPSCALE, RESTORE, ANIMATE), each routed to its own engine. Compound requests are split so no half is silently dropped; steps the LLM invents are pruned deterministically. Masks are chosen by the best available evidence and validated with pure geometry (floor, ceiling, speckle, subject-leak against a BiRefNet matte) — the face is structurally protected. Verification measures seams and scores realism, prompt accuracy, identity, consistency and artifacts; failures escalate through parameters, then model, then workflow, and the best attempt is kept. Flux Kontext's silent refusals are detected by measuring whether the image changed at all, and rerouted to inpainting.

49 validated workflow templates back these routes (*backend/app/workflows/*): generation (*draft/hires/XL/Z-Image/Kontext/portrait*), *img2img* and *style*, *inpaint* (*universal*, *hires*, *remove*, *replace-background*), *outpaint*, *upscale* (*model-based* and *creative*), *relight*, *pose*, *angles*, *compose*, *identity*, *face detail*, *restore*, *sketch-to-photo*, *video* (*t2v*, *i2v*, *inpaint*, *outpaint*), *motion transfer* (*standard* and *fast*), *3D reconstruction* and *scene3d*. The planner can also author new graphs, which are schema-checked against the LIVE ComfyUI node list before execution, with one self-repair round.

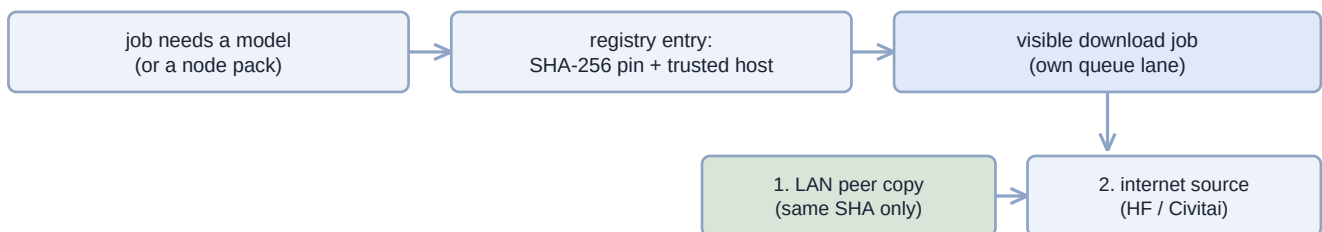
6. Jobs and the three-lane queue



Source: `backend/app/core/jobs.py`

Twelve job types (`image_edit`, `workflow`, `video`, `motion_transfer`, `avatar`, `avatar_render`, `model_download`, `model_research`, `node_pack`, `setup`, `discover`, `update`) run through one visible queue with pause, reorder, cancel and retry. Failures retry with exponential backoff; transient engine deaths revive the engine and continue. Downloads live on their own lane so a multi-gigabyte fetch never delays a render — and a downloading machine still reports itself idle to peers. ETAs come from the median of past run times of the same kind.

7. Models: registry, search, trust

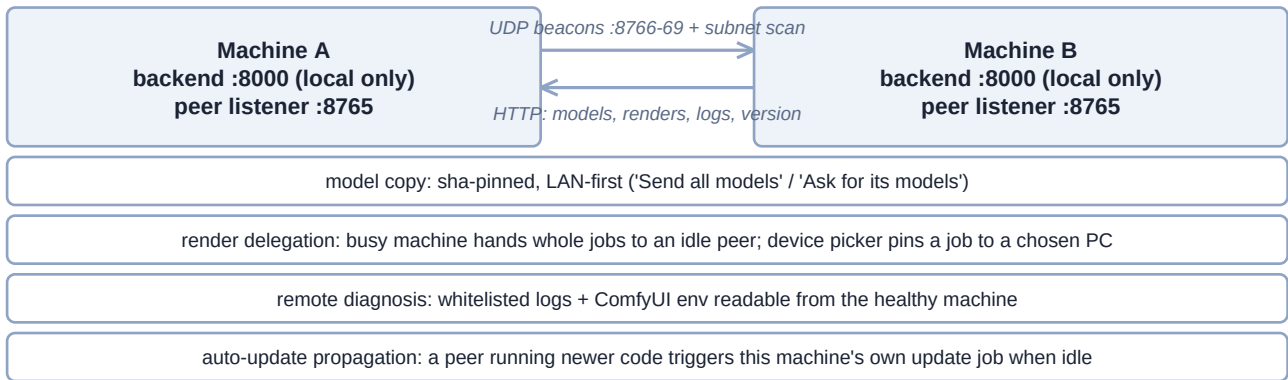


bytes are checksum-verified either way; unknown or NSFW-flagged sources are never auto-installed (`trust.py`); missing Ollama models auto-pull; missing curated node packs auto-in

Sources: `backend/app/core/registry.py`, `trust.py`, `model_search.py`, `model_intel.py`, `node_packs.py`

35 curated registry entries cover checkpoints, video models, text encoders, VAEs, LoRAs, ControlNets, upscalers and segmentation weights, each with a SHA-256 pin and a typed destination folder that ComfyUI reads directly. When a prompt needs something new, the scout searches Hugging Face and Civitai, a trust layer vets the source (safetensors only, known hosts, pickle gate, adoption floor; NSFW-flagged sources are never auto-installed), and gated files self-heal to public mirrors only when the published hash matches byte-for-byte. A model knowledge file (`data\model_knowledge.json`) records what each checkpoint is good at and steers model choice. Seven curated ComfyUI node packs (`impact-pack`, `controlnet-aux`, `frame-interpolation`, `rmbg`, `ic-light`, `instantid`, `gguf`) install as visible jobs — automatically when a request needs one — and are verified against the live node list after restart.

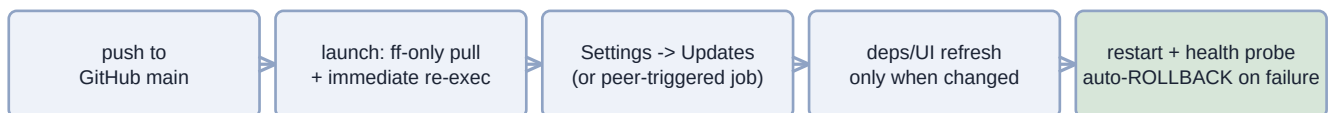
8. Two or more machines: the LAN fabric



Source: `backend/app/core/peers.py`; firewall rules via `allow-lan.ps1` (TCP 8765, UDP 8766-8769, private networks only)

Machines find each other with UDP beacons (multi-interface, directed broadcasts) plus an active subnet scan, and appear live in the UI rail with GPU, VRAM and RAM. The app itself stays on 127.0.0.1 — only the peer listener is exposed. All transfers are checksum-verified against this machine's own pins; renders proxied to a busy peer wait politely instead of dying; a peer running newer code triggers this machine's own ordinary update job when idle.

9. Updates



data\ is untracked - an update can never touch models, photos or the database; dirty local edits block updates by name

Sources: `launch.ps1` section 0, `backend/app/core/update.py`

10. Safety and privacy

All policy lives in one user-owned module, `backend/app/core/safety.py`: it screens prompts, gates avatar creation behind explicit consent, and blocks NSFW-flagged model sources from auto-installing. Hard lines — minors, photo-undressing of real people, non-consensual imagery — are enforced regardless of any toggle. Settings offers a custom rule manager whose rules are read live on every check. Local-first is structural: the cloud LLM is only a fallback and every reply that used it is stamped as such.

11. API reference (all under /api on 127.0.0.1:8000)

Area	Endpoints
Health & system	GET /health - GET /system - GET /events - DELETE /events - DELETE /history/prompts
Assets & gallery	POST/GET /assets - GET /assets/{id}/file - DELETE /assets/{id} - POST /assets/{id}/restore - GET /gallery - DELETE /gallery - GET /versions/{id}/file - POST /versions/{id}/promote
Editing	POST /edits - POST /edits/plan - POST /masks/preview - POST /masks/point - GET /masks/status
Generation & video	POST /workflows/generate - POST /workflows/run - POST /video - POST /motion_transfer
Avatars	POST /avatar - GET /avatars - DELETE /avatars/{id} - POST /avatars/{id}/render
Jobs & queue	GET /jobs - GET /jobs/{id} - POST /jobs/{id}/cancel retry move - DELETE /jobs/{id} - POST /jobs/clear - GET /queue/state - POST /queue/pause resume
Models	GET /models - POST /models/{name}/download - GET /models/search files index civitai - POST /models/propose propose-civitai - GET /nodepacks - POST /nodepacks/{name}/install
LAN peers	GET /peers - POST /peers/probe - POST /peers/push-models - POST /peers/fetch-models - GET /peers/log
Updates & settings	GET /update - POST /update/apply - GET/POST /settings - GET/POST /safety/rules - DELETE /safety/rules/{id}
Workflow discovery	POST /workflows/discover - POST /workflows/approve

Source: `backend/app/api/routes.py` (60 routes)

12. Configuration (environment variables)

Variable	Meaning (default)
PROMPTFORGE_DATA_DIR	where models/photos/DB live (data\)
PROMPTFORGE_COMFYUI_DIR / _URL	ComfyUI install dir (set by the launcher) / API url (:8188)
PROMPTFORGE_LLM_URL / _MODEL / _API_MODEL	local LLM endpoint (:11434), planner model, cloud fallback
PROMPTFORGE_INPAINT_BACKEND / _SEGMENT_BACKEND	comfyui mock / sam mock - mock means fully offline
PROMPTFORGE_CRITIC_MODEL / _MIN / _RETRIES	vision judge (llava), minimum score, retry count
PROMPTFORGE_QUALITY_ROUNDS / _TARGET	edit-loop iterations (2) and target score (95)
PROMPTFORGE_ADHERENCE_ROUNDS / _TARGET	prompt-adherence escalation ladder settings
PROMPTFORGE_AUTO_INSTALL / _FIRST_RUN_SETUP	allow automatic model/pack downloads (on) / first-run scout
PROMPTFORGE_LAN_SHARE / _RENDER / _PEER_HOSTS / _PEER_AUTO_UPDATE	LAN model sharing (on), accept peer renders (on), static peer list, follow newer peers (on)
PROMPTFORGE_AUTO_UPDATE	launcher git auto-update (on; 0 pins the machine to its current version)
PROMPTFORGE_JOB_MAX_RETRIES / _JOB_BACKOFF_S	queue retry policy
PROMPTFORGE_MAX_UPLOAD_MB / _MAX_VIDEO_SECONDS	upload and clip limits
PROMPTFORGE_CIVITAI_TOKEN	unlocks token-gated Civitai downloads (never logged)
PROMPTFORGE_WORKFLOW_REPAIRS	LLM graph self-repair rounds

Source: `backend/app/config.py`

13. On-disk layout and logs

Path	What lives there
backend\	Flask app + core (its .venv is created on first launch)

Path	What lives there
frontend\dist\	built UI the backend serves (rebuilt when sources change)
tools\ComfyUI\	auto-installed render engine (an existing install elsewhere is found and reused, including venv-only ROCm layout)
data\models\	typed model folders ComfyUI loads directly (checkpoints, diffusion_models, text_encoders, vae, loras, controlnet, ...)
data\gallery, data\promptforge.db	images/videos and the database (assets, jobs, experience, model knowledge)
data\logs\	launch.log (full launcher transcript), comfyui[-err]-install[-repair].log, directml-install.log, sage/xformers/sam-install.log
doctor.ps1	read-only end-to-end health check -> data\logs\doctor-report.txt
allow-lan.ps1	one-time firewall opening for the LAN features (run as admin on both PCs)

This document is generated by docs\generate_documentation.py — if it disagrees with the code, regenerate it; if it still disagrees, the generator is the bug.